FUNDAMENTOS DE APRENDIZAGEM DE MÁQUINA

Maikon Lucian Lenz



SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Aprendizagem supervisionada: aplicação de árvores para classificação e regressão

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Desenvolver um conjunto de dados para as tarefas de predição, classificação ou regressão.
- Aplicar algoritmos de árvore de decisão.
- Determinar métricas para avaliação dos resultados obtidos utilizando métodos de amostragem e validação.

Introdução

As árvores de decisão dividem o espaço de hipóteses, seja para rotular a saída ou para determinar valores contínuos de saída. Esse método, apesar de não garantir uma solução ótima global, é bastante utilizado e costuma apresentar bom desempenho tanto em classificação quanto em regressão. Assim como outros algoritmos de aprendizado, a biblioteca do Scikit-Learn implementa métodos e classes que facilitam o desenvolvimento de aplicações utilizando esses modelos.

Neste capítulo, você vai ver de que forma as árvores de decisão podem ser parametrizadas usando o Scikit-Learn e vai conhecer a importância das métricas de avaliação e das técnicas de amostragem, em especial, o *hold-out* e o *cross-validation*.

1 Árvores de decisão e regressão no Scikit-Learn

As árvores de decisão são algoritmos não paramétricos que, como o nome sugere, fazem uso de regras de decisão para determinar um atributo alvo a partir dos atributos previsores. O treinamento de uma árvore de decisão consiste no aprendizado dessas regras de decisão. Trata-se, portanto, de um método baseado em procura, quando um espaço de hipóteses é varrido para encontrar a melhor solução (FACELI *et al.*, 2011).

A biblioteca Scikit-Learn, por sua vez, implementa o algoritmo de árvores de decisão, assim como outros métodos de aprendizado. Nela, há um módulo destinado exclusivamente a esses algoritmos: `sklearn.tree` (SCIKIT--LEARN, c2019a).

Os algoritmos de decisão produzem grafos acíclicos, no sentido de um nó folha ou de outras divisões do espaço de hipóteses em que os nós de divisão expressam um teste condicional dos valores de atributos previsores. Como critério de aprendizado, são usadas funções de custo 0-1 para obter a moda ou minimizar a média do erro médio quadrático, para tarefas de classificação e regressão, respectivamente (FACELI *et al.*, 2011). A Figura 1 ilustra uma árvore de decisão e o respectivo espaço de hipóteses.

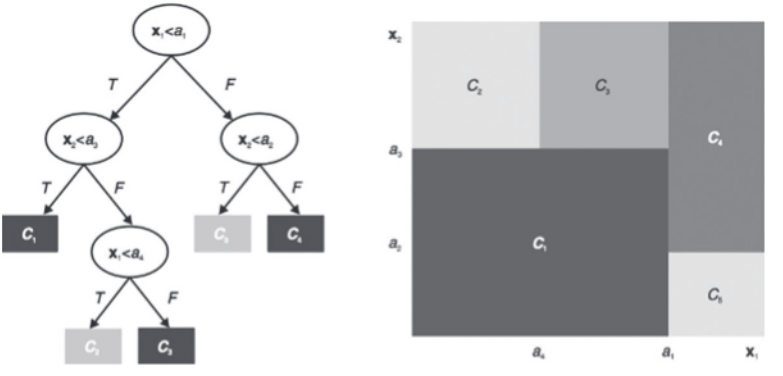


Figura 1. Árvore de decisão (à esquerda) e o respectivo espaço de hipóteses (à direita).
Fonte: FACELI *et al.* (2011, p. 84).

Enquanto a função de custo não satisfazer o critério, o espaço de hipóteses será recursivamente subdividido, conforme a Figura 1. As decisões não podem ser reconsideradas e o algoritmo pode levar a uma solução ótima local, porém, com solução normalmente rápida e satisfatória. Os atributos que serão alvo do teste de decisão em cada nó são determinados segundo a entropia, ou seja, o teste será utilizado para aqueles que representam menor aleatoriedade na variável do alvo (FACELI *et al.*, 2011).

Além da função de custo, opções que impactam fortemente sobre o desempenho de uma árvore de decisão são os critérios de poda, que podem ser pré-poda, para evitar o crescimento de alguns nós, ou pós-poda, para remover nós que não contribuem de forma significativa para a acurácia de classificação. É necessário também ter um cuidado especial com valores desconhecidos, uma vez que a árvore de decisão é hierárquica e testes deixarão de ser feitos a partir do momento que um atributo nulo apareça (FACELI *et al.*, 2011).

Os algoritmos mais populares de árvores de decisão são quatro: ID3, que utiliza o atributo categórico com maior ganho de informação para a construção da árvore, que é expandida até o seu tamanho máximo e, somente ao final, podada para aumentar a capacidade de generalização; C4.5, uma evolução do ID3 que permite o uso de atributos não categóricos utilizando valores discretizados, cuja saída também difere do ID3 por não ser mais uma árvore, mas um conjunto de regras de decisão; C5.0, que não apresenta modificações substanciais, porém, desempenho e acurácia superiores ao C4.5; e CART (do inglês, *classification and regression trees*). O CART, implementado pela biblioteca Scikit-Learn, pode ser utilizado tanto para regressões quanto classificações e se assemelha ao algoritmo C4.5, contudo, permite o uso de variáveis numéricas e constrói a árvore ao invés do conjunto de regras de decisão (SCIKIT-LEARN, c2019a).

Dois métodos são utilizados: o `sklearn.tree.DecisionTreeClassifier`, para tarefas de classificação, e o `sklearn.tree.DecisionTreeRegressor`, para tarefas de regressão. O Quadro 1 apresenta os parâmetros dos modelos de árvore de decisão.

Quadro 1. Parâmetros dos modelos de árvore de decisão

Parâmetro	`DecisionTree-Classifier`	`DecisionTree-Regressor`
criterion — `'gini'`: impureza Gini; `'entropy'`: entropia/aleatoriedade.	X	
criterion — `'mse'`: erro quadrático médio; `'friedman_mse'`: aprimoramento de Friedman para o mse; `'mae'`: erro médio absoluto.		X
splitter — `'best'`: melhor divisão; `'random'`: divisão aleatória.	X	X
max_depth — profundidade máxima (`None`).	X	X
min_samples_split — número mínimo de amostras para dividir um nó (`2`);	X	X
min_samples_leaf – número mínimo de amostras em um nó folha (`1`);	X	X
min_weight_fraction_leaf — menor peso total para um nó folha (`0,0`);	X	X
max_features — quantidade máxima de características relevantes para dividir um nó (`None`).	X	X
random_state — número de base para criar números aleatórios (`None`);	X	X
max_leaf_nodes — número máximo de nós folha (`None`);	X	X
min_impurity_decrease — menor redução aceitável do indicador de impureza para dividir um nó (`0,0`);	X	X

(Continua)

(Continuação)

Quadro 1. Parâmetros dos modelos de árvore de decisão

Parâmetro	`DecisionTree-Classifier`	`DecisionTree-Regressor`
min_impurity_split — maior valor aceitável de impureza para um nó folha (`1,0e-7`);	X	X
class_weight — `'balanced'`, lista ou dicionário contendo os pesos (`None`).	X	
ccp_alpha — custo de complexidade mínimo (`0,0`).	X	X

A maioria dos parâmetros e valores padrões assumidos por cada um dos modelos são comuns em ambos os casos, com exceção do parâmetro `criterion`, que dispõe de opções diferentes para função de custo do classificador e regressor, e do parâmetro `class_weight`, que obviamente não está presente no regressor por este lidar apenas com alvo de valor contínuo (SCIKIT-LEARN, c2019b, c2019c).

2 Algoritmos de árvore de decisão

Um classificador do tipo árvore de decisão pode ser inicializado utilizando o código-fonte a seguir.

```
# Exemplo 1 - Classificador
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
```

Assim como em outros algoritmos da biblioteca Scikit-Learn, estão presentes os métodos `fit`, `predict` e `predict_proba`. O método `fit` recebe um conjunto de dados separado em atributos previsores (parâmetro `X`) e atributo alvo (parâmetro `y`) e é utilizado no treinamento do modelo, como no exemplo apresentado a seguir.

```
# Exemplo 2 - Treinamento
# importa o método load_iris
from sklearn.datasets import load_iris

# carrega os dados separando em atributos previsores (X) e alvo (y)
X, y = load_iris(return_X_y=True)
# treina o modelo de árvore de decisão
model.fit(X=X, y=y)
```

Nesse exemplo, é utilizado o conjunto de dados Iris da biblioteca Scikit-Learn. Nele, 150 amostras de flores são classificadas em três espécies diferentes a partir de suas características de comprimento e largura das sépalas e pétalas. Para tanto, o método `load_iris` carrega o conjunto de dados e, por meio do parâmetro `return_X_y` como `True`, retorna o conjunto na forma de uma tupla, dividindo-o em atributos previsores (variável `X`) e alvo (variável `y`). Na sequência, as variáveis `X` e `y` alimentam o método `fit` do modelo instanciado.

Para comparar o conjunto de dados original e seus respectivos rótulos com os rótulos previstos pelo modelo treinado, são criados `DataFrame`, que facilitam a substituição de valores numéricos por categorias, como é o caso da coluna espécies, e também tornam mais prático seu uso nas demais ferramentas, como Seaborn para produzir gráficos. No `DataFrame` criado, cada coluna representa um atributo e o atributo previsor tem seus valores dados pelos nomes das espécies conforme o dicionário criado para mapeá-los. Veja a seguir.

```
# Exemplo 3 - Previsão, DataFrames e Exibição de Gráficos
# Bibliotecas para criar DataFrame e gráficos
import pandas as pa
import seaborn as sb
import matplotlib.pyplot as pyp

# Dicionário que mapeará os rótulos numéricos para respectivas
espécies
dic = {0: 'Setosa', 1: 'Versicolor', 2: 'Virginica'}
```

```
# DataFrame do conjunto original
df = pa.DataFrame(X, columns=['Comprimento das Sépalas',
                              'Largura das Sépalas',
                              'Comprimento das Pétalas',
                              'Largura das Pétalas'])
df['Espécie'] = y
df['Espécie'] = df['Espécie'].apply(lambda i: dic[i])

# Previsão dos rótulos a partir dos atributos previsões
y_prevs = model.predict(X)

# DataFrame com os rótulos previstos
df_prevs = df.copy()
df_prevs['Espécie'] = y_prevs
df_prevs['Espécie'] = df_prevs['Espécie'].apply(lambda i:
dic[i])

# Gráficos de distribuição dos dados originais e previstos
fig, axes = pyp.subplots(1, 2, figsize=(10,4))
sb.scatterplot(      x=df['Comprimento das Pétalas'],
                     y=df['Comprimento das Sépalas'],
                     hue=df['Espécie'], ax=axes[0])
sb.scatterplot(      x=df_prevs['Comprimento das Pétalas'],
                     y=df_prevs['Comprimento das Sépalas'],
                     hue=df_prevs['Espécie'],
                     ax=axes[1])
axes[0].set_title('a) Rótulos originais.')
axes[1].set_title('b) Rótulos previstos.')
```

Ao final desse exemplo, são criadas duas colunas de imagem para exibir a distribuição de dados conforme os rótulos originais (Figura 2a) e os rótulos previstos (Figura 2b) utilizando o modelo treinado e o método `predict` com os valores de atributo previsor `X`. Uma vez que o conjunto de dados possui quatro atributos previsores, dois deles são selecionados para criar os gráficos, já que, apesar de possível, fazer um gráfico de quatro dimensões e um gráfico de três dimensões tornaria mais difícil comparar as instâncias.

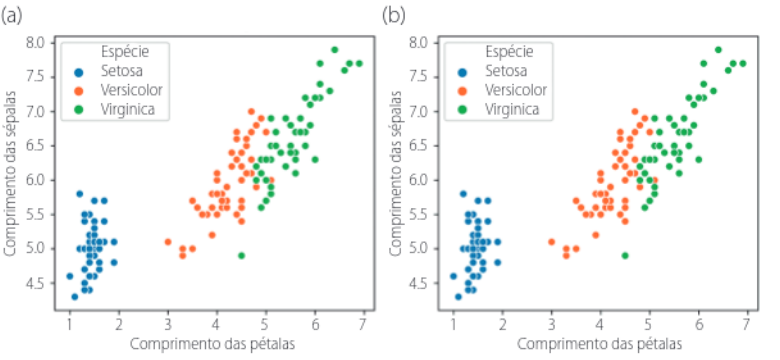


Figura 2. (a) Distribuição dos dados conforme rótulos originais. (b) Distribuição dos dados conforme rótulos previstos.

Percebe-se que o modelo foi capaz de prever todos os rótulos corretamente. Ainda é possível visualizar a árvore gerada a partir de um método `plot_tree` disponível no módulo `sklearn.tree` conforme o exemplo 4.

```
# Exemplo 4 - Plot _ tree
# Importa a biblioteca tree
import sklearn.tree

# Cria uma área de gráfico de tamanho específico
fig, ax = pyp.subplots(figsize=(16, 8))
# Plota a árvore preenchida pela cor da classe que predomina no nó
arvore = sklearn.tree.plot _ tree(model, max _ depth=1000, font-
size=10, filled=True, ax=ax)
```

O resultado pode ser visualizado na Figura 3. Percebe-se que apenas um teste é necessário para separar uma das três espécies das demais, produzindo um nó folha já no primeiro nível de profundidade. Dali em diante, para separar as outras duas espécies, são necessários mais quatro níveis. Esse distanciamento para uma das classes já era esperado, uma vez que nos gráficos da Figura 2 é possível notar que uma das classes está agrupada com uma separação bem definida das demais.

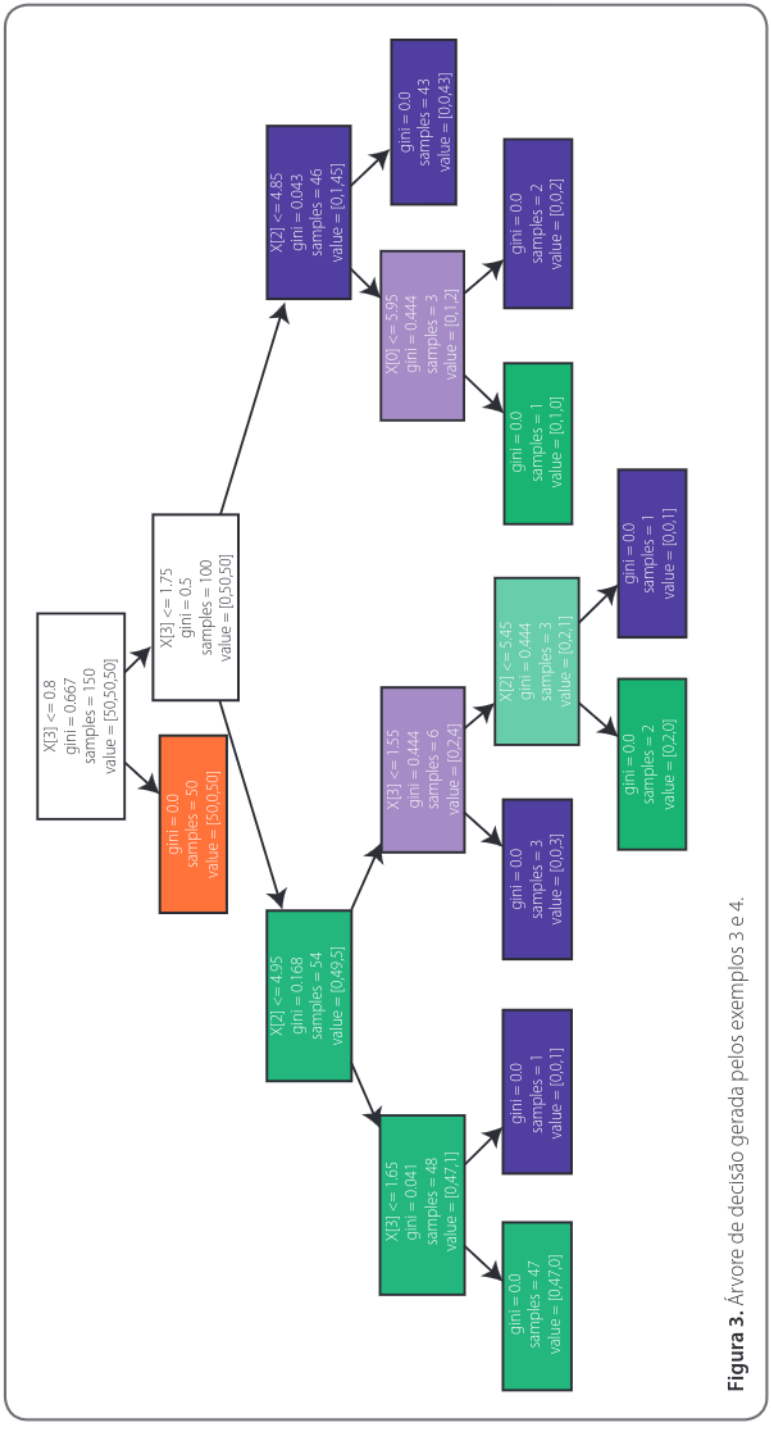


Figura 3. Árvore de decisão gerada pelos exemplos 3 e 4.

A forma como as árvores de regressão é criada obedece a procedimento similar, mas utilizando, dessa vez, o `sklearn.tree.DecisionTreeRegression`. No exemplo 5, um modelo é treinado a partir de um conjunto de dados aleatório utilizando a biblioteca Numpy para criar uma distribuição senoidal de dados de um único período (2π).

```
# Exemplo 5 - Senoide
# Importa método de regressão e a biblioteca Numpy
from sklearn.tree import DecisionTreeRegressor
import numpy as np

# Cria uma senoide
XR = np.sort(2*np.pi * np.random.rand(50, 1), axis=0)
yR = np.sin(XR).ravel()

# Plota a senoide destacando as instâncias
pyp.plot(XR, yR, 'ro-')
```

A função `np.random.rand` produz 50 números aleatórios entre 0 e 1 que, multiplicados por 2π, irão criar diversos pontos de uma senoide completa ao longo do eixo X. Então, os valores são ordenados pelo método `np.sort` para facilitar o desenho da linha que seguirá de 0 até 2π. Os valores de `yR` resultam simplesmente da função seno dos valores de `XR`. A Figura 4 ilustra a distribuição dos dados.

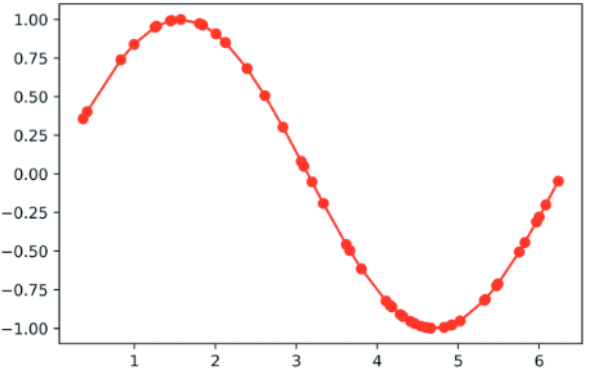


Figura 4. Senoide produzida com o auxílio da biblioteca Numpy e de funções trigonométricas e aleatórias.

Na sequência, o exemplo 6 cria dois modelos de árvore de regressão com níveis de profundidade 2 e 4. Os modelos são treinados e utilizam o atributo previsor original para prever o atributo alvo e comparar por meio do gráfico a distribuição original e cada uma das previsões.

```
# Exemplo 6 - Treinamento com dois níveis de profundidade
diferentes
              E exibição dos resultados.
# Cria um novo modelo
modelR2 = DecisionTreeRegressor(max_depth=2)
modelR4 = DecisionTreeRegressor(max_depth=4)

# treina o modelo de árvore de decisão
modelR2.fit(X=XR, y=yR)
modelR4.fit(X=XR, y=yR)

# Previsão dos rótulos a partir dos atributos previsões
yR2_prevs = modelR2.predict(XR)
yR4_prevs = modelR4.predict(XR)

fig, ax = pyp.subplots(figsize=(12,8))
pyp.plot(XR, yR, 'ro-', label='Original')
pyp.plot(XR, yR2_prevs, 'bx-', label='Regressão Profundidade
Máxima: 2')
pyp.plot(XR, yR4_prevs, 'gx-', label='Regressão Profundidade
Máxima: 4')
pyp.legend()
```

A proximidade de cada curva em relação à original pode ser observada na Figura 5, em que as linhas azul e verde correspondem à árvore de regressão com profundidade máxima 2 e 4, respectivamente.

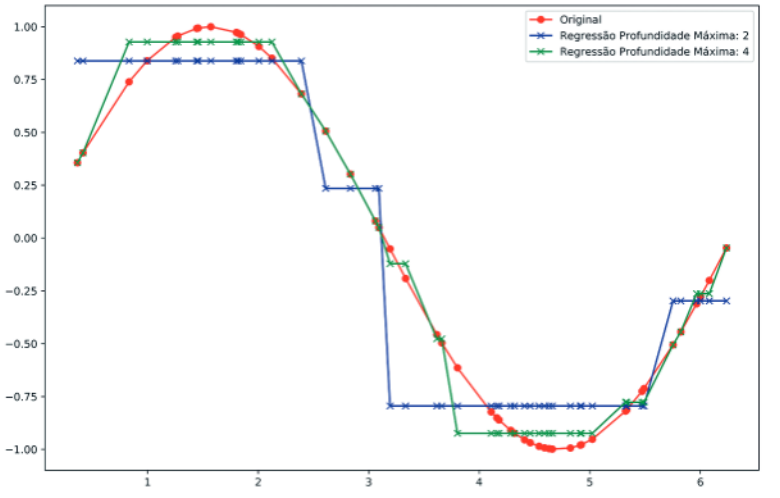


Figura 5. Proximidade das regressões comparadas ao conjunto de dados original em vermelho.

Assim como no caso da classificação, pode-se plotar as árvores criadas a partir do treinamento para diferentes profundidades configuradas utilizando o exemplo 7.

```
# Exemplo 7 - Árvores de Regressão
fig, ax = pyp.subplots(2, 1, figsize=(16, 10))
arvoreR2 = sklearn.tree.plot_tree(modelR2, fontsize=8,
filled=True,
ax=ax[0])
arvoreR4 = sklearn.tree.plot_tree(modelR4, fontsize=8,
filled=True,
ax=ax[1])
ax[0].set_title('a) Profundidade: 2')
ax[1].set_title('b) Profundidade: 4')
```

As árvores são exibidas na Figura 6, sendo a primeira para profundidade 2 (Figura 6a) e a segunda para a profundidade 4 (Figura 6b).

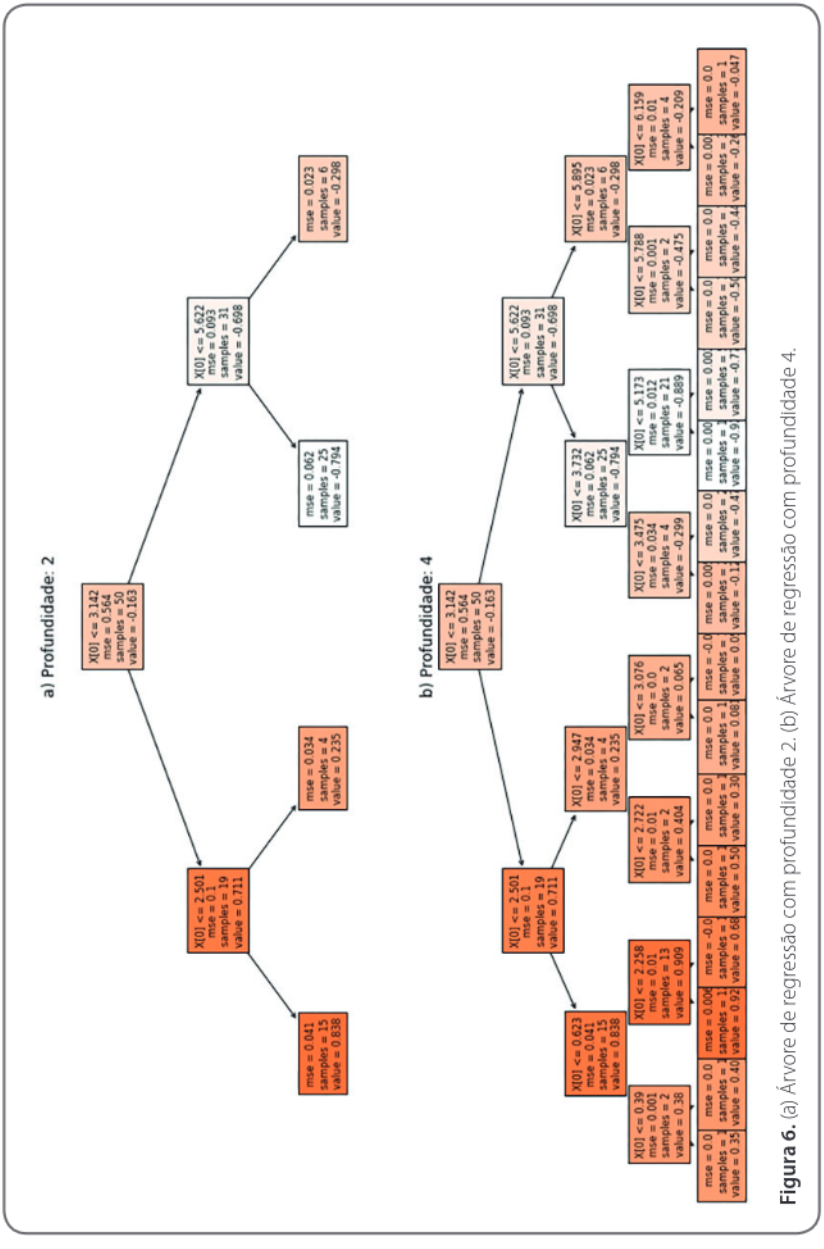


Figura 6. (a) Árvore de regressão com profundidade 2. (b) Árvore de regressão com profundidade 4.

Percebe-se que, qualquer que seja a configuração, em ambos os algoritmos, seja classificação ou regressão, uma árvore será construída para prever o atributo alvo. Em outras palavras, um mínimo local sempre é possível. Entretanto, cabe utilizar métricas de avaliação para definir a parametrização mais adequada para que a árvore de decisão possa generalizar adequadamente e manter alto índice de acurácia.

3 Amostragem e avaliação

Para encontrar a melhor parametrização, é preciso utilizar métricas de avaliação. Em geral, somente é possível verificar a eficácia de um modelo durante o teste medindo taxas de erro e acerto (FACELI *et al.*, 2011).

Em tarefas de aprendizado supervisionado, como é o caso das árvores de decisão e regressão, é possível comparar o previsto a partir de um atributo previsor com o atributo alvo original e verificar a margem de erro, se for uma regressão, ou acurácia, se for classificação. Além disso, nem sempre o maior acerto significará o melhor modelo, pois, facilmente, um algoritmo de aprendizado de máquina pode se tornar superadaptado e perder sua capacidade de generalização. Em outras palavras, o modelo prevê com muita eficiência as mesmas instâncias para as quais foi treinado, mas é incapaz de fazer boas previsões para instâncias até então desconhecidas.

Uma forma de diminuir o potencial de superadaptação é treinar o modelo a partir de amostragens ao invés de utilizar o conjunto inteiro de dados como treinamento. Duas técnicas comuns de amostragem são: *hold-out*, em que o conjunto de dados é dividido em treinamento e teste, para uma menor proporção de conjunto de treinamento, mas que ainda mantém boa representatividade dos dados; e o *cross-validation*, validação cruzada, em inglês, em que o conjunto de dados é dividido em vários grupos de mesmo tamanho e cada grupo serve uma vez como conjunto de teste, enquanto os demais são utilizados como treinamento.

No *hold-out*, uma vez definida a proporção adequada, o treinamento ocorre uma única vez e é mais afetado pela variação do conjunto de dados. No *cross--validation*, o treinamento é efetuado tantas vezes quanto a quantidade de divisões criadas no conjunto de dados e o que importa é o valor médio entre todos os treinamentos.

As duas técnicas contam com métodos próprios na biblioteca do Scikit--Learn. O exemplo 8 aplica a técnica de *hold-out* e treina um modelo de árvore de decisão para nove proporções diferentes de treinamento/teste.

```
# Exemplo 8 - Hold-out
from sklearn.model _ selection import train _ test _ split
from sklearn.metrics import accuracy _ score

Xh, yh = load _ iris(return _ X _ y=True)

acc = []
tt = np.arange(0.1, 1, 0.1)
for t in tt:
    X _ treino, X _ teste, y _ treino, y _ teste = train _ test _
split(Xh, yh,
                                          test _ size=t,
                                          random _ state=4)
    modelh = DecisionTreeClassifier(random _ state=4)
    modelh.fit(X=X _ treino, y=y _ treino)
   acc.append(accuracy _ score(y _ teste,    modelh.
predict(X _ teste))*100)

pa.DataFrame(acc, index=tt*100, columns=['Acurácia'])
```

Para facilitar a análise, uma tabela foi montada com os valores de acurácia medidos com a iteração com amostragem diferente, relacionando-os com a proporção do conjunto de testes, conforme a Figura 7.

	Acurácia
10.0	93.333333
20.0	96.666667
30.0	97.826087
40.0	96.666667
50.0	93.333333
60.0	93.333333
70.0	96.226415
80.0	97.500000
90.0	94.074074

Figura 7. Acurácia obtida para diferentes proporções de amostragem.

Percebe-se que a acurácia aumenta até 30% quando atinge o maior valor entre todas as amostras. Era provável que esses resultados variassem bastante devido à aleatoriedade com que os grupos foram formados. Se aplicada a técnica de *cross-validation*, provavelmente, resultados mais constantes seriam observados. O algoritmo do exemplo 9 implementa essa técnica.

```
# Exemplo 9 - Cross-validation
from sklearn.model_selection import cross_val_score

Xc, yc = load_iris(return_X_y=True)

acc = []
dfc = pa.DataFrame()
for r in range(1, 5, 1):
    modelc = DecisionTreeClassifier(random_state=r)
    s = 'Acurácia R' + str(r)
    cross = cross_val_score(modelc, Xc, yc, cv=10,
                                    scoring='accuracy')*100
    cross = np.append(cross, sum(cross)/10)
    dfc[s] = cross
dfc
```

Nesse caso, cada uma das linhas se refere a uma iteração do *cross-validation*, com exceção da última linha (10), que é composta pelo valor médio. É feito um laço para testar a variação dos índices com a aleatoriedade das amostragens alterando o valor de `random_state`, conforme a Figura 8.

	Acurácia R1	Acurácia R2	Acurácia R3	Acurácia R4
0	100.000000	100.000000	100.000000	100.000000
1	93.333333	93.333333	93.333333	93.333333
2	100.000000	100.000000	100.000000	100.000000
3	93.333333	93.333333	93.333333	93.333333
4	93.333333	93.333333	93.333333	93.333333
5	86.666667	86.666667	86.666667	86.666667
6	93.333333	93.333333	93.333333	93.333333
7	93.333333	100.000000	100.000000	100.000000
8	100.000000	100.000000	100.000000	100.000000
9	100.000000	100.000000	100.000000	100.000000
10	95.333333	96.000000	96.000000	96.000000

Figura 8. Acurácia para diferentes estados de `random_state`.

Percebe-se pela tabela que, independentemente da aleatoriedade, a acurácia média permaneceu sempre próxima de 96%, sendo um indicador mais confiável do desempenho do modelo.

Tanto o *hold-out* quanto o *cross-validation* são utilizados em tarefas de regressão. Entretanto, as métricas de avaliação são outras, já que o alvo, nesse caso, é um valor contínuo. Se a acurácia é uma das métricas mais utilizadas em classificadores, o erro quadrático médio desempenha papel similar nos regressores (FACELI *et al.*, 2011).

O exemplo 10 aplica o *hold-out* para o mesmo conjunto de dados criado no exemplo 6.

```
# Exemplo 10 – Erro Quadrático Médio (MSE, Mean Squared Error)
from sklearn.metrics import mean _ squared _ error

Xh = np.sort(2*np.pi * np.random.rand(50, 1), axis=0)
yh = np.sin(Xh).ravel()

mse = []
tt = np.arange(0.1, 1, 0.1)
for t in tt:
    X _ treino, X _ teste, y _ treino, y _ teste = train _ test _
split(Xh, yh,
                                test _ size=t, random _ state=4)
    modelh = DecisionTreeRegressor(random _ state=4)
    modelh.fit(X=X _ treino, y=y _ treino)
    mse.append(mean _ squared _ error(y _ teste,
                                 modelh.predict(X _ teste))*100)
pa.DataFrame(mse, index=tt*100, columns=['MSE'])
```

Ao contrário da acurácia, busca-se pelo erro quadrático médio obter o menor valor possível. Nesse caso, a proporção de 10% de instâncias para o conjunto de testes apresentou o melhor desempenho, conforme a Figura 9. Entretanto, assim como na classificação, o *hold-out* é bastante impactado pela aleatoriedade das amostras.

	MSE
10.0	6.666667
20.0	3.333333
30.0	2.173913
40.0	3.333333
50.0	6.666667
60.0	6.666667
70.0	3.773585
80.0	2.500000
90.0	5.925926

Figura 9. Erro quadrático médio para diferentes proporções de amostragem.

O exemplo 11, por sua vez, utiliza o modelo de regressão em conjunto com o método de *cross-validation*. Destaca-se do método `cross_val_score` em relação ao exemplo 9 quanto ao parâmetro `scoring`, antes igual a `'accuracy'`, agora, igual a `'neg_mean_squared_error'`.

```
# Exemplo 11 - CV - Erro Quadrático Médio (MSE, Mean Squared
Error)
from sklearn.model_selection import cross_val_score

Xc = np.sort(2*np.pi * np.random.rand(50, 1), axis=0)
yc = np.sin(Xc).ravel()

dfc = pa.DataFrame()
for d in range(2, 10, 1):
    modelc = DecisionTreeRegressor(max_depth=d)
    s = 'MSE D=' + str(d)
    cross = cross_val_score(modelc, Xc, yc, cv=10,
                    scoring='neg_mean_squared_error')*100
    cross = np.append(cross, sum(cross)/10)
    dfc[s] = cross
dfc
```

Dessa vez, é testada a variação do parâmetro de profundidade máxima (`max_depth`), de 2 a 9, e se percebe uma estabilização do erro a partir de uma profundidade igual a 4, conforme a Figura 10. É preciso lembrar que a última linha (10) indica a média de todas as amostragens para uma mesma coluna de parametrização.

	MSE D=2	MSE D=3	MSE D=4	MSE D=5	MSE D=6	MSE D=7	MSE D=8	MSE D=9
0	-24.155813	-14.851149	-7.613967	-7.613967	-7.613967	-7.613967	-7.613967	-7.613967
1	-0.661231	-3.850198	-2.191539	-2.171343	-1.569444	-1.410351	-1.354740	-1.354740
2	-7.076914	-2.194151	-1.100899	-1.416064	-0.669858	-0.439986	-0.439986	-0.439986
3	-5.370231	-1.273698	-4.198073	-2.012920	-1.551306	-1.551029	-1.551029	-1.551029
4	-4.050692	-3.310500	-3.574125	-2.979229	-2.706954	-2.706954	-2.706954	-2.706954
5	-26.327970	-8.019630	-2.407475	-2.031443	-2.031443	-2.031443	-2.031443	-2.031443
6	-9.693887	-3.475195	-3.951768	-2.834197	-2.521406	-2.521406	-2.521406	-2.521406
7	-7.121416	-2.758469	-0.427354	-1.559234	-0.795005	-0.829872	-0.840499	-0.840499
8	-7.629665	-3.758616	-4.923632	-3.776841	-3.668869	-3.631642	-3.615998	-3.615998
9	-45.216117	-19.034341	-19.034341	-19.034341	-19.034341	-19.034341	-19.034341	-19.034341
10	-13.730393	-6.252595	-4.942317	-4.542958	-4.216259	-4.177099	-4.171036	-4.171036

Figura 10. Erro quadrático médio para diferentes níveis de profundidade máxima da árvore.

Os classificadores e os regressores possuem muitas métricas disponíveis para a avaliação. Enquanto um classificador irá visar à avaliação de erro e acerto usando acurácia, índice de sensibilidade, especificidade, precisão e assim por diante, todos dependentes dos verdadeiros/falsos e positivos/negativos, um regressor irá utilizar medições contínuas, como erro quadrático, erro absoluto, desvio e assim por diante.

Referências

FACELI, K. *et al. Inteligência artificial*: uma abordagem de aprendizado de máquina. Rio de Janeiro: LTC, 2011.

SCIKIT-LEARN. *Decision trees*. c2019a. Disponível em: https://scikit-learn.org/stable/modules/tree.html. Acesso em: 24 abr. 2020.

SCIKIT-LEARN. *sklearn.tree.DecisionTreeClassifier*. c2019b. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html. Acesso em: 24 abr. 2020.

SCIKIT-LEARN. *sklearn.tree.DecisionTreeRegressor*. c2019c. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html. Acesso em: 24 abr. 2020.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS